

Strategy Creation using MACD

Rules :

- 1. BUY when MACD line crosses above Signal line
- 2. SELL when MACD line crosses below Signal line

```
In [1]: # Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import yfinance as yf
import datetime
import quantstats as qs
import warnings
warnings.filterwarnings("ignore")
```

```
In [26]: class BacktestingMACD:

    def __init__(self, ticker , start , end ):
        self.ticker = ticker
        self.start = start
        self.end = end
        self.fetch_data()
        self.indicator()
        self.signal()
        self.position()
        self.returns()

    def fetch_data(self):
        self.data = yf.download(self.ticker , self.start , self.end)
        self.data = self.data.droplevel(1 , axis = 1)

    def indicator(self):
        ## MACD Line creation
        self.data["ema12"] = self.data["Close"].ewm(span = 12).mean()
        self.data["ema26"] = self.data["Close"].ewm(span = 26).mean()
        self.data["MACD_line"] = self.data["ema12"] - self.data["ema26"]

        ## Signal Line creation
        self.data["signal_line"] = self.data["MACD_line"].ewm(span = 9).mean()

    def signal(self): ## here we are generating signals

        self.data["signal"] = np.where((self.data["MACD_line"] > self.data["signal_line"])&
                                       (self.data["MACD_line"].shift(1) < self.data["signal_line"].shift(1)), 1 , 0)

        self.data["signal"] = np.where((self.data["MACD_line"] < self.data["signal_line"])&
                                       (self.data["MACD_line"].shift(1) > self.data["signal_line"].shift(1)), -1 , self.data["signal"])

    def position(self):

        self.data["position"] = self.data["signal"].replace(to_replace=0 , method = "ffill")

    def returns(self):

        self.data["bnh"] = np.log(self.data["Close"] / self.data["Close"].shift(1))

        self.data["strategy_returns"] = self.data["position"].shift(1) * self.data["bnh"]
        print("The strategy returns:" , np.round(self.data["strategy_returns"].cumsum()[-1], 2))
        return self.data["strategy_returns"].cumsum()[-1]

    def analysis(self):

        self.data[["MACD_line" , "signal_line", "position"]].plot(figsize = (15,9) , secondary_y = "position" ,grid = True)
        self.data[["bnh" , "strategy_returns"]].cumsum().plot(figsize = (15,9) , grid = True)

        qs.reports.basic(self.data["strategy_returns"]])
```

```
In [27]: ticker = "BTC-USD"
end = datetime.date(2026,9,20)
start = end - pd.Timedelta(days = 365 * 1)
```

```
In [28]: a = BacktestingMACD(ticker , start , end)

[*****100%*****] 1 of 1 completed
The strategy returns: 0.43
```

```
In [29]: a.analysis()
```

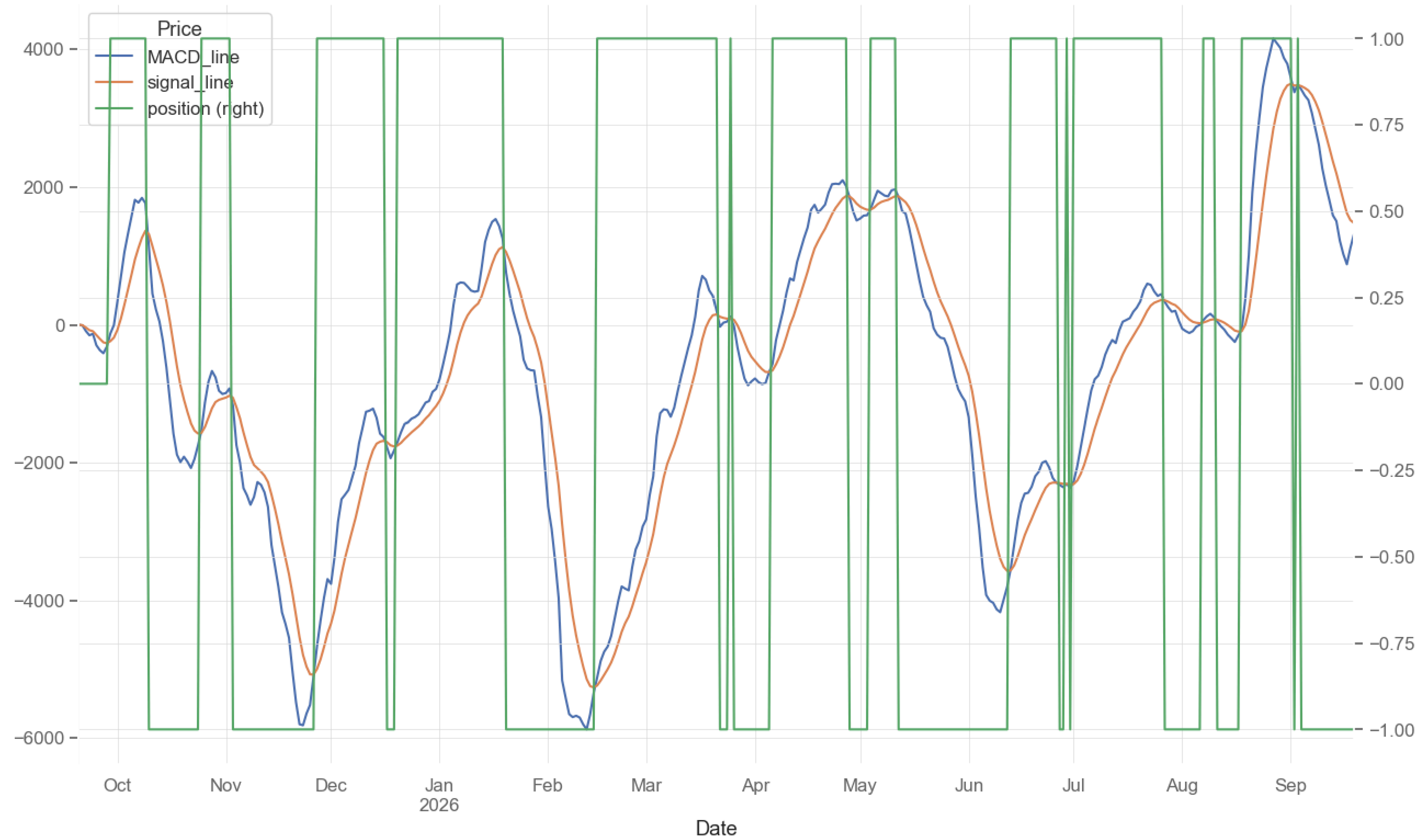
Performance Metrics

Parameter	Value
Risk-Free Rate	0.0%
Periods/Year	252
Compounded	Yes
Match Dates	Yes

	Strategy
Start Period	2025-09-21
End Period	2026-09-19
Risk-Free Rate	0.0%
Time in Market	98.0%
Cumulative Return	39.67%
CAGR %	26.03%
Sharpe	0.81
Prob. Sharpe Ratio	83.71%
Sortino	1.27
Sortino/v2	0.9
Omega	1.16
Max Drawdown	-19.65%
Max DD Date	2026-04-05
Max DD Period Start	2026-02-06
Max DD Period End	2026-06-01
Longest DD Days	116
Gain/Pain Ratio	0.16
Gain/Pain (1M)	1.61
Payoff Ratio	1.1
Profit Factor	1.16
Common Sense Ratio	1.6
CPC Index	0.65
Tail Ratio	1.39
Outlier Win Ratio	4.05
Outlier Loss Ratio	3.04

MTD	-10.41%
3M	-2.71%
6M	8.73%
YTD	40.16%
1Y	39.67%
3Y (ann.)	26.03%
5Y (ann.)	26.03%
10Y (ann.)	26.03%
All-time (ann.)	26.03%
Avg. Drawdown	-10.06%
Avg. Drawdown Days	42
Recovery Factor	2.2
Ulcer Index	0.11
Serenity Index	0.49

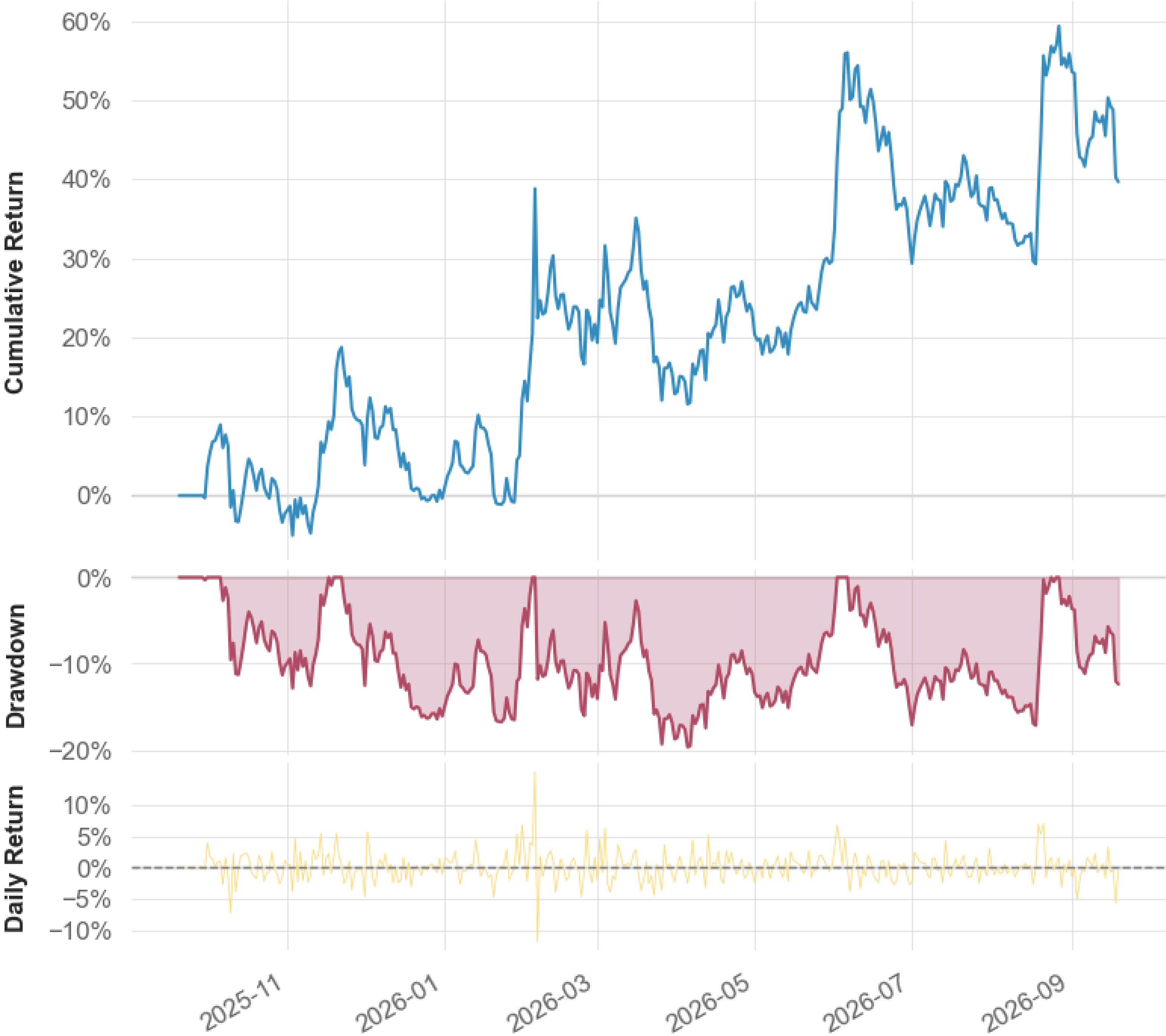
Strategy Visualization



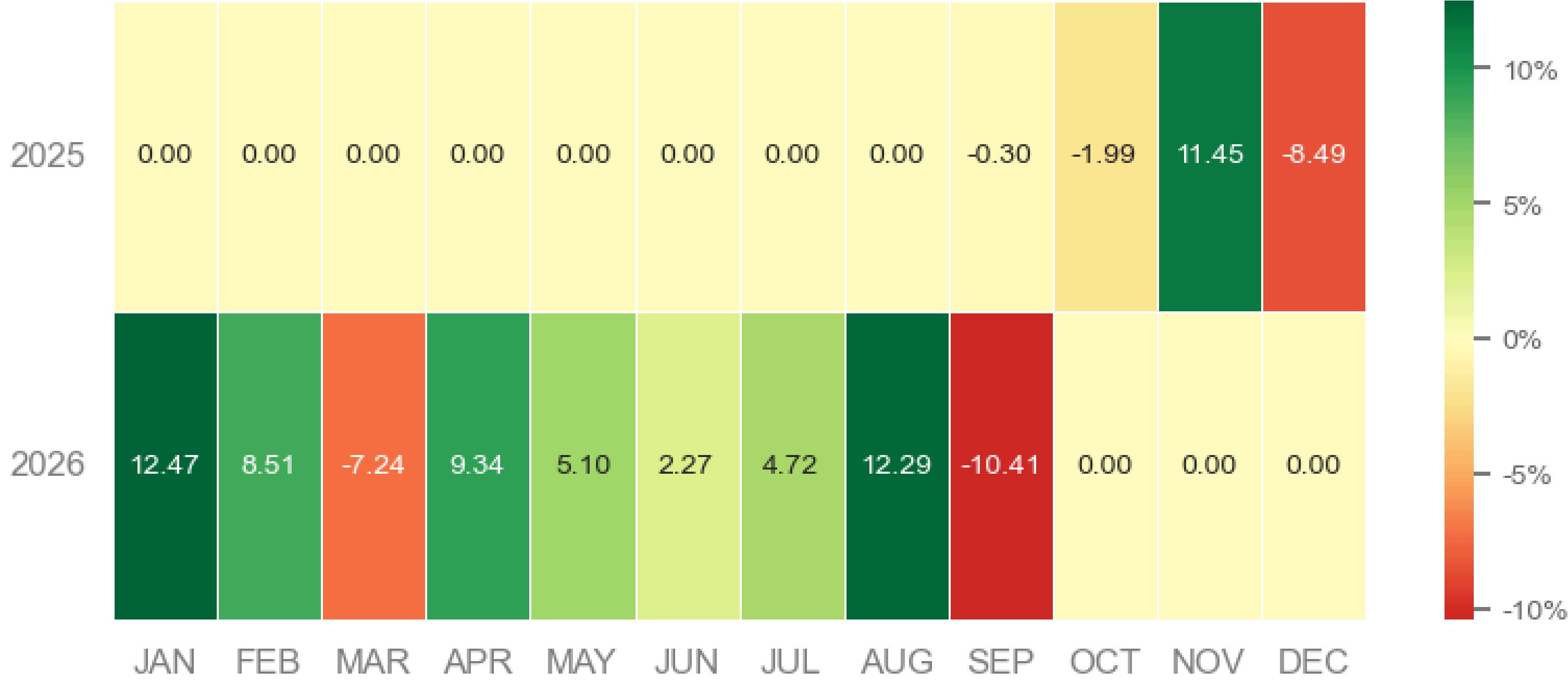


Portfolio Summary

20 Sep '25 - 19 Sep '26 ; Sharpe: 0.81



Strategy - Monthly Returns (%)



Now lets check for differnt assets.

```
In [31]: stock_list = [ "AAPL",
    "TSLA",
    "RELIANCE.NS",
    "INFY.NS",
    "SPY",
    "BTC-USD",
    "ETH-USD",
    "GC=F",
    "CL=F",
    "EURUSD=X"]

stock = []
returns = []
```

```
for x in stock_list:
    print("For stock :", x)
    a = BacktestingMACD(x , start ,end)
    stock.append(x)
    returns.append(a.returns())
```

[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed

For stock : AAPL
The strategy returns: 0.23
The strategy returns: 0.23
For stock : TSLA
The strategy returns: -0.19
The strategy returns: -0.19
For stock : RELIANCE.NS
The strategy returns: -0.1
The strategy returns: -0.1
For stock : INFY.NS

[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed

The strategy returns: 0.1
The strategy returns: 0.1
For stock : SPY
The strategy returns: -0.04
The strategy returns: -0.04
For stock : BTC-USD
The strategy returns: 0.43
The strategy returns: 0.43
For stock : ETH-USD
The strategy returns: 0.26
The strategy returns: 0.26
For stock : GC=F
The strategy returns: 0.04
The strategy returns: 0.04
For stock : CL=F

[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed

The strategy returns: -0.41
The strategy returns: -0.41
For stock : EURUSD=X
The strategy returns: 0.03
The strategy returns: 0.03

```
In [33]: outcome = pd.DataFrame({"stock" : stock , "returns" : returns})
outcome.sort_values(by="returns" , ascending=False)
```

Out[33]:

	stock	returns
5	BTC-USD	0.432901
6	ETH-USD	0.258611
0	AAPL	0.228278
3	INFY.NS	0.100507
7	GC=F	0.038770
9	EURUSD=X	0.029553
4	SPY	-0.041276
2	RELIANCE.NS	-0.099466
1	TSLA	-0.191177
8	CL=F	-0.411089

```
In [ ]:
```